Inteligencia Artificial I · Curso 2026-27

Práctica 5 · Sistemas de reglas y lógica difusa

«¿A qué partida te mando?» — el emparejador de un juego competitivo

Sesión	Martes 27 de octubre, 1 hora, aula 9 (presentación, dudas y arranque de la Parte A)
Entrega	Viernes 6 de noviembre, 23:59, en el Campus Virtual
Organización	Por parejas (las mismas de las prácticas anteriores)
Material	P5_material_alumno.zip (Python 3.9+; numpy y matplotlib solo para las figuras)

1. Objetivo

Cuando pulsáis «buscar partida» en un juego competitivo por equipos (League of Legends, Valorant, Rocket League…), un sistema decide en segundos contra quién vais a jugar. Usa lo que sabe de vosotros: ranking, ratio de victorias de las últimas partidas, rol que jugáis, si interactuáis por el chat, cuánto lleváis esperando en la cola… y también, según algunas patentes, si gastáis dinero en la tienda. Con todo eso os asigna a una partida pareja, contra mejores que vosotros o contra peores.

En esta práctica vais a construir ese emparejador dos veces, con las dos técnicas del Tema 5: como sistema basado en reglas con factores de certeza (los hechos discretos: compra, chatea, rol fijo, racha) y como controlador difuso (las variables continuas: ratio y ranking). Programaréis los dos motores desde cero, en dos ficheros pequeños, y comprobaréis que reproducís exactamente los ejemplos de clase antes de aplicarlos al emparejador.

No hay que instalar ningún motor externo (Drools, JESS, CLIPS). Todo el código es vuestro; el esqueleto solo fija la representación de los datos. El fichero `verificar.py` comprueba 25 valores sacados de las transparencias y del material: **ejecutadlo antes de entregar**.

2. Parte A · Reglas con factores de certeza

Fichero `fc.py`. Un hecho es una cadena y lleva asociado su factor de certeza en [−1, 1]; una regla es un diccionario con el antecedente (lista de literales, conjunción), el consecuente (un hecho) y el FC de la regla. El literal `"no X"` se evalúa con FC(¬X) = −FC(X); la función `fc_literal` ya está hecha.

- **A1. Fórmulas básicas.** Implementad `fc_negacion`, `fc_conjuncion` (mínimo) y `fc_disyuncion` (máximo).
- **A2. Propagación.** `fc_regla(fc_premisa, fc_de_la_regla)`: FC(h) = FC(e) · FC(h/e) si FC(e) > 0, y 0 en otro caso.
- **A3. Acumulación de evidencias.** `combinar(fc1, fc2)` con la fórmula de la transparencia 74 (tres casos según los signos).
- **A4. Encadenamiento hacia adelante.** `encadenar_adelante(hechos, reglas, traza)`: recorre las reglas en el orden dado y dispara cada una **una sola vez**, en cuanto todas sus premisas son conocidas. Si la premisa es ≤ 0 la regla no aporta nada; si la hipótesis ya tenía un FC, se combina con el nuevo aporte. Devuelve un diccionario **nuevo** (no modifiquéis el de entrada). Con `traza=True` debe imprimir una línea por regla, como en el ejemplo del final del fichero. Comprobación: el ejercicio de la transparencia 71 da FC(H) = 0,303 y el de la 75 da FC(C) = −0,432.

A5. **Ampliad el emparejador.** `base_matchmaking.py` trae 15 reglas en dos niveles (observaciones → perfil del jugador → tipo de partida) y tres jugadores de consulta: uno en racha, uno frustrado y un Diamante que lleva mucho en cola. Ejecutadlo y leed la traza de los tres. Después añadid **al menos un hecho observable nuevo** (p. ej. ha sido reportado, juega en grupo con amigos, es su primera semana) y **dos reglas que converjan sobre una misma hipótesis**, una a favor y otra en contra (FC negativo), y un cuarto jugador en el que ambas disparen. En la memoria pegad la traza de ese jugador y explicad, paso a paso, de dónde sale el FC final de cada tipo de partida.

A6. **El orden importa.** Tomad el jugador 2 pero con `compra = -0,5`. Con el orden original `contra_peores` sale positivo; si movéis R10 delante de R7, sale negativo: el jugador cambia de partida solo por reordenar la base. Explicad por qué ocurre con este motor y qué habría que cambiar en `encadenar_adelante` para que el resultado no dependiera del orden. No hace falta implementarlo; sí razonarlo.

A7. **¿Debe el emparejador saber si pagas?** R11 manda a quien gasta dinero y está frustrado contra rivales peores. Riot afirma que su emparejamiento usa solo el nivel y el rol; Activision patentó en 2017 un emparejador diseñado para fomentar las compras. En diez líneas: ¿qué puede hacer técnicamente un SBR con ese dato, qué debería hacer, y qué ventaja tiene aquí un sistema de reglas frente a una ponderación opaca? (Pista: ¿quién puede pedirle explicaciones a cada uno?)

3. Parte B · Lógica difusa

Fichero `difusa.py`. Una variable lingüística es un universo numérico y un diccionario de etiquetas, cada una con su triángulo (a, b, c): vale 0 fuera de [a, c] y 1 en b. Una regla es la conjunción de pares (variable, etiqueta) y concluye una etiqueta de una variable de salida. La función `evaluar` (ciclo completo) ya está escrita: solo llama a las vuestras.

B1. **Triangular.** `triangular(x, a, b, c)`. Se admite a = b o b = c para las etiquetas de los extremos (hombros): fijaos en que las comparaciones de los extremos tienen que ser estrictas, y explicad en la memoria qué pasaría con un hombro si fueran x ≤ a y x ≥ c como en la transparencia 101.

B2. **Fuzzificación.** `fuzzificar(x, variable)` devuelve el grado de pertenencia de x a cada etiqueta. Comprobación: Laura (179 cm) con la variable `ALTURA` da pequeño 0, mediano 0,08, alto 0,32.

B3. **Inferencia mín-máx.** `inferir(reglas, grados)`: el grado de cada regla es el **mínimo** de sus antecedentes; cuando varias reglas concluyen la misma etiqueta se queda el **máximo**.

B4. **Centroide.** `centroide(variable, grados, paso)`: cada etiqueta de salida se trunca a su grado, se agregan con el máximo y se calcula Σ μ(y)·y / Σ μ(y) sobre el universo discretizado. Si ninguna etiqueta está activa devuelve el centro del universo. Comprobación: con pequeño 0,5, mediano 0,3 y alto 0 sale 158,1 cm (transparencia 120).

B5. **El emparejador difuso.** `emparejador.py` decide el **ajuste** (diferencia de MMR del rival buscado, de −500 a +500) a partir del **ratio** de victorias y el **ranking**, con cinco reglas; dos de ellas aplanan la salida hacia «parejos» en los extremos, porque un Challenger no tiene mejores contra quien jugar ni un Hierro IV tiene peores. Ejecutadlo, ejecutad `python superficie.py emparejador` y comentad la superficie en tres líneas: ¿qué variable manda? ¿dónde se nota cada regla?

B6. **Vuestro controlador.** En `mi_emparejador.py` diseñad un **segundo** controlador del emparejador con **dos entradas, una salida, tres etiquetas por variable y entre 4 y 6 reglas**. Elegid vosotros las variables: tiempo en cola, racha de derrotas seguidas, ping, horas jugadas esta semana, tamaño del grupo con el que entráis…; y la salida: amplitud de búsqueda en puntos de MMR, prioridad en cola, tiempo máximo de espera… Ejecutad `python superficie.py mi_emparejador` e incluid en

la memoria las dos figuras (funciones de pertenencia y superficie de control) y una tabla con cinco puntos de prueba y su salida, al menos uno de ellos con dos reglas activas a la vez.

B7. **Tres preguntas.** (a) Por qué el ajuste del emparejador nunca llega a ±500 aunque una etiqueta de hombro esté activa al 100 % (calculad el centroide de «peores» truncada a 1). (b) Qué cambia si en `inferir` sustituís el mínimo por el producto: probadlo en (ratio 80 %, ranking 2800) y dad los dos valores del ajuste. (c) Para el punto de vuestro controlador con dos reglas activas, seguid el cálculo a mano hasta los grados de salida y comparadlo con lo que imprime el programa.

4. Entregables

Un único fichero `apellido1_apellido2_lab5.zip` con:

- `fc.py`, `base_matchmaking.py` (ampliada), `difusa.py` y `mi_emparejador.py`. El resto de ficheros del material no hace falta devolverlos.
- Las dos figuras PNG de vuestro controlador.
- La memoria en PDF, **cuatro páginas como máximo**, con estas secciones: (1) salida completa de `python verificar.py` pegada tal cual; (2) ampliación del emparejador con la traza explicada (A5), y respuestas a A6 y A7; (3) comentario de la superficie del emparejador (B5) y vuestro controlador: variables, reglas, figuras, tabla de puntos (B6); (4) respuestas a B7; (5) quién ha hecho qué.

5. Evaluación

Criterio	Peso
Parte A: funciones y motor (A1–A4), según verificar.py y lectura del código	20 %
Ampliación del emparejador y preguntas sobre el orden y las compras (A5–A7)	20 %
Parte B: funciones (B1–B4), según verificar.py y lectura del código	20 %
Emparejador difuso comentado y controlador propio: diseño, figuras, puntos (B5, B6)	25 %
Memoria y preguntas (B7): claridad, corrección, honestidad con lo que no funciona	15 %

Un fichero que no pasa alguna comprobación de `verificar.py` no se penaliza por sí mismo si la memoria dice cuál falla y por qué; lo que se penaliza es entregarlo sin mencionarlo. El controlador propio se valora por el diseño y la discusión, no por lo rebuscado de las variables: tres etiquetas bien puestas y cinco reglas razonadas valen más que diez reglas copiadas de un tutorial. En la sesión del martes 27 se mostrará una demo interactiva del emparejador (reglas y difusa) con la misma base de conocimiento que este material; el enlace estará en el Campus Virtual.

Anexo · Fórmulas que se usan

Operación	Fórmula
Negación	$FC(\neg x) = -FC(x)$
Conjunción	$FC(A \text{ y } B) = \min[FC(A), FC(B)]$
Disyunción	$FC(A \text{ o } B) = \max[FC(A), FC(B)]$
Propagación por una regla	$FC(h) = FC(e) \cdot FC(h/e)$ si $FC(e) > 0$; 0 si $FC(e) \leq 0$
Acumulación (dos reglas)	$FC1 + FC2 - FC1 \cdot FC2$ si ambos > 0; $FC1 + FC2 + FC1 \cdot FC2$ si ambos < 0; $(FC1 + FC2) / (1 - \min(\lvert FC1 \rvert, \lvert FC2 \rvert))$ en otro caso
Triangular (a, b, c)	0 si $x < a$ o $x > c$; $(x-a)/(b-a)$ si $a \leq x < b$; 1 si $x = b$; $(c-x)/(c-b)$ si $b < x \leq c$
Inferencia Mamdani	grado de la regla = min de los antecedentes; grado de la etiqueta = max de sus reglas
Centroide	$y^* = \Sigma\, \mu(y) \cdot y \,/\, \Sigma\, \mu(y)$, con $\mu(y)$ = max sobre etiquetas de min(grado, μ_etiqueta(y))